III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 1

AIM

To compute the measures of **central tendency** (Mean, Median, Mode) and **dispersion** (Variance, Standard Deviation, Range) for a given dataset using Python.

THEORY

Central Tendency

Central Tendency gives a single representative value for the entire dataset.

Measure	Meaning
Mean	Average of all values
Median	Middle value of sorted data
Mode	Most frequently occurring value

Dispersion

Dispersion tells us how spread out the data is.

Measure	Meaning
Variance	Average of squared differences from the mean
Standard Deviation (SD)	Square root of variance (same unit as data)
Range	Difference between max & min value

CODE

```
# Experiment 1: Central Tendency & Dispersion
# Slightly extended version for lab record

import numpy as np
from scipy import stats

# Sample Data
data = [10, 20, 20, 30, 40, 50, 50, 60]

# --- Central Tendency ---
mean = np.mean(data)
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
median = np.median(data)
mode = stats.mode(data, keepdims=True)[0][0]

# --- Dispersion ---
variance = np.var(data)
std_dev = np.std(data)
range_val = max(data) - min(data)   # ADDED (optional but useful)

# Print Output
print("\n=== DATA ===")
print(data)

print("\n=== CENTRAL TENDENCY ===")
print("Mean =", mean)
print("Median =", median)
print("Mode =", mode)

print("\n=== DISPERSION ===")
print("Variance =", variance)
print("Standard Deviation =", std_dev)
print("Range =", range_val)
```

OUTPUT

```
=== DATA ===
[10, 20, 20, 30, 40, 50, 50, 60]

=== CENTRAL TENDENCY ===
Mean = 35.0
Median = 35.0
Mode = 20

=== DISPERSION ===
Variance = 275.0
Standard Deviation = 16.583123951777
Range = 50
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 2

◈ AIM

To apply basic data preprocessing techniques on a dataset including:

1. Attribute (feature) selection
2. Handling missing values
3. Discretization
4. Outlier removal

◈ THEORY

1 Attribute Selection

Selecting only the useful columns/features from the dataset.
Used to:

- Reduce model complexity
- Improve accuracy
- Reduce computation time

2 Handling Missing Values

Missing values cause incorrect results.
Common techniques:

- Remove missing rows
- Replace with mean/median/mode
- Use interpolation

3 Discretization

Converting continuous data into categorical data.
Example:
90 → "High", 45 → "Medium"

Used for:

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

- simplification
- decision trees
- histogram-based learning

4 Outlier Removal

Outliers are values that are **too far from normal range**.
Detected using:

- IQR
- Z-score

Used to:

- Improve accuracy
- Avoid distortion of mean

◈ CODE :

```
import pandas as pd
import numpy as np

# Sample Dataset
data = {
    'Age': [18, 22, 25, 120, np.nan, 30, 29, 100],
    'Marks': [80, 85, np.nan, 75, 95, 92, 88, 91],
    'Dept': ["CSE", "ECE", "ECE", "CSE", "IT", "CSE", np.nan, "ECE"]
}

df = pd.DataFrame(data)
print("Original Data:\n", df)

# --- Handling Missing Values ---
df['Marks'].fillna(df['Marks'].mean(), inplace=True)
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Dept'].fillna(df['Dept'].mode()[0], inplace=True)

# --- Outlier Removal (using IQR) ---
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1
df = df[(df['Age'] >= Q1 - 1.5 * IQR) & (df['Age'] <= Q3 + 1.5 * IQR)]

# --- Attribute Selection ---

df_selected = df[['Age', 'Marks']]
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
# --- Discretization ---
df_selected['Marks_Category'] = pd.cut(df_selected['Marks'],
                                       bins=[0, 70, 85, 100],
                                       labels=["Low", "Medium",
"High"])

print("\nAfter Preprocessing:\n", df_selected)
```

OUTPUT :

```
Original Data:
    Age  Marks Dept
0  18.0  80.0  CSE
1  22.0  85.0  ECE
2  25.0   NaN  ECE
3 120.0  75.0  CSE
4   NaN  95.0   IT
5  30.0  92.0  CSE
6  29.0  88.0  NaN
7 100.0  91.0  ECE

After Preprocessing:
    Age  Marks Marks_Category
0  18.0  80.00        Medium
1  22.0  85.00          High
2  25.0  86.00          High
5  30.0  92.00          High
6  29.0  88.00          High
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 3

◈ AIM

To apply the **K-Nearest Neighbors (KNN) algorithm** for:

1. Classification
2. Regression

using a real-world dataset.

◈ THEORY :

◈ KNN Algorithm

KNN is a **supervised learning algorithm** that makes predictions based on the **closest K neighbors**.

How KNN Works

1. Store training data
2. For a new input:
    - Calculate distance (Euclidean)
    - Find K nearest points
    - Predict output using neighbors (majority vote or average)

Used For:

✓ Classification
✓ Regression
✓ Recommendation systems
✓ Pattern recognition

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

DATASET :

Dataset represents students with:

- Hours studied
- Sleep hours
- Score/Pass status

CODE :

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.metrics import accuracy_score, mean_squared_error

# ----------------------------
# SAMPLE DATASET
# ----------------------------
data = {
    "Hours_Studied": [2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Sleep_Hours":  [9, 8, 7, 7, 6, 5, 5, 4, 3],
    "Score":        [50, 55, 60, 65, 70, 78, 85, 90, 95],
    "Pass":         [0, 0, 0, 1, 1, 1, 1, 1, 1]
}

df = pd.DataFrame(data)
print("\n=== DATASET ===")
print(df)

# ----------------------------
# SPLITTING DATA
# ----------------------------
X = df[['Hours_Studied', 'Sleep_Hours']]
y_class = df['Pass']     # For classification
y_reg = df['Score']      # For regression

X_train, X_test, y_train_cls, y_test_cls = train_test_split(X, y_class,
test_size=0.3)
_, _, y_train_reg, y_test_reg = train_test_split(X, y_reg,
test_size=0.3)

# ----------------------------
# KNN CLASSIFICATION
# ----------------------------
knn_cls = KNeighborsClassifier(n_neighbors=3)
knn_cls.fit(X_train, y_train_cls)
pred_cls = knn_cls.predict(X_test)
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
print("\nClassification Accuracy:", accuracy_score(y_test_cls,
pred_cls))

# ----------------------------
# KNN REGRESSION
# ----------------------------
knn_reg = KNeighborsRegressor(n_neighbors=3)
knn_reg.fit(X_train, y_train_reg)
pred_reg = knn_reg.predict(X_test)
print("\nRegression MSE:", mean_squared_error(y_test_reg, pred_reg))

print("\nPredicted Scores:", pred_reg)
```

◈ SAMPLE OUTPUT :

```
=== DATASET ===
   Hours_Studied  Sleep_Hours  Score  Pass
0              2            9     50     0
1              3            8     55     0
2              4            7     60     0
3              5            7     65     1
4              6            6     70     1
5              7            5     78     1
6              8            5     85     1
7              9            4     90     1
8             10            3     95     1

Classification Accuracy: 1.0
Regression MSE: 25.3
Predicted Scores: [73.3 88.0 63.3]
```

(Sample — your output will differ based on train/test split)

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 4

◈ AIM

To demonstrate the **Decision Tree algorithm** for a classification problem and perform **parameter tuning** for better results.

◈ THEORY

📌 Decision Tree

A decision tree is a tree-like structure where:

- Each **internal node** = feature test
- Each **branch** = outcome
- Each **leaf node** = final class/decision

Advantages

✓ Easy to understand & visualize
✓ Handles both numerical & categorical data
✓ No complex scaling / normalization needed

Parameter Tuning Examples

- `max_depth` (depth of tree)
- `criterion` ("gini" or "entropy")
- `min_samples_split`

Common Use Cases

✓ Medical diagnosis
✓ Loan approval
✓ Fraud detection
✓ Customer churn prediction

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

DATASET

We will predict whether a person **buys a computer** based on:

- Age
- Income
- Student status

CODE

```
# Decision Tree Classification with Parameter Tuning

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# ----------------------------
# Sample Dataset
# ----------------------------
data = {
    "Age": [25, 30, 45, 35, 22, 40, 28, 48],
    "Income": ["High", "Medium", "High", "Medium", "Low", "Low",
"Medium", "High"],
    "Student": ["No", "Yes", "No", "Yes", "Yes", "No", "Yes", "No"],
    "Buys_Computer": ["No", "Yes", "Yes", "Yes", "No", "No", "Yes",
"Yes"]
}

df = pd.DataFrame(data)

# Convert categorical variables
df = pd.get_dummies(df, drop_first=True)

# Split data
X = df.drop("Buys_Computer_Yes", axis=1)
y = df["Buys_Computer_Yes"]

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3)

# ----------------------------
# Base Model (No tuning)
# ----------------------------
dt1 = DecisionTreeClassifier()
dt1.fit(X_train, y_train)
pred1 = dt1.predict(X_test)
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
# ----------------------------
# Tuned Model
# ----------------------------
dt2 = DecisionTreeClassifier(max_depth=3, criterion="entropy")
dt2.fit(X_train, y_train)
pred2 = dt2.predict(X_test)

print("Accuracy (Without Tuning):", accuracy_score(y_test, pred1))
print("Accuracy (With Tuning):", accuracy_score(y_test, pred2))
```

OUTPUT

```
Accuracy (Without Tuning): 0.50
Accuracy (With Tuning): 1.0
```

(*Exact values change due to train/test split*)

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 5

◈ AIM

To demonstrate the **Decision Tree Regression** algorithm on a real-world dataset and evaluate its performance.

◈ THEORY

📌 Decision Tree Regression

A decision tree regressor predicts **continuous numeric values** by splitting data into regions and assigning output values to each region.

How it Works

1. Create splits based on feature values
2. Minimize error (MSE or MAE)
3. Assign average value to each leaf node

Advantages

✓ No need for scaling
✓ Can model non-linear relationships
✓ Very easy to interpret

Disadvantages

✗ Can overfit
✗ Not very stable (small changes affect results)

◈ USE CASE

We will predict **house prices** based on:

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

- Area (sq ft)
- Bedrooms
- Age of the building (years)

CODE :

```
# Decision Tree Regression

import pandas as pd
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# ------------------------------
# Sample Dataset
# ------------------------------
data = {
    'Area': [1500, 1800, 2400, 3000, 3500, 4000, 600, 1200],
    'Bedrooms': [3, 4, 3, 5, 4, 5, 2, 3],
    'Age': [5, 10, 8, 15, 20, 12, 3, 6],
    'Price': [35, 45, 50, 65, 70, 80, 20, 30]  # price in lakhs
}

df = pd.DataFrame(data)
print("\n=== DATASET ===")
print(df)

# ------------------------------
# Split Data
# ------------------------------
X = df[['Area', 'Bedrooms', 'Age']]
y = df['Price']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3)

# ------------------------------
# Decision Tree Regressor
# ------------------------------
tree = DecisionTreeRegressor(max_depth=3)
tree.fit(X_train, y_train)

pred = tree.predict(X_test)

mse = mean_squared_error(y_test, pred)

print("\nPredicted Prices:", pred)
print("Actual Prices:", y_test.tolist())
print("Mean Squared Error:", mse)
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

SAMPLE OUTPUT

```
=== DATASET ===
   Area  Bedrooms  Age  Price
0  1500         3    5     35
1  1800         4   10     45
2  2400         3    8     50
3  3000         5   15     65
4  3500         4   20     70
5  4000         5   12     80
6   600         2    3     20
7  1200         3    6     30

Predicted Prices: [50 70 35]
Actual Prices: [50, 70, 35]
Mean Squared Error: 0.0
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 6

AIM

To apply the **Random Forest algorithm** for both **classification and regression** and evaluate its performance on a real-world dataset.

THEORY

Random Forest

Random Forest is an **ensemble learning algorithm** that builds multiple decision trees and combines their results.

How it Works

1. Creates multiple decision trees using bootstrapped data
2. Uses random subset of features
3. Takes majority vote (classification) or average prediction (regression)

Advantages

✓ High accuracy
✓ Handles missing values well
✓ Resistant to overfitting
✓ Works well for both classification & regression

Disadvantages

✗ Slower than single decision tree
✗ Less interpretable

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

USE CASE

Dataset: Predict whether a customer will buy a product **(Classification)** and predict customer spending amount **(Regression)**.

CODE

```
# Random Forest Classification and Regression
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier,
RandomForestRegressor
from sklearn.metrics import accuracy_score, mean_squared_error

# ----------------------------
# Sample Dataset
# ----------------------------
data = {
    "Age": [22, 25, 30, 35, 40, 45, 50, 60],
    "Income": [25000, 30000, 40000, 45000, 60000, 80000, 100000,
120000],
    "Credit_Score": [580, 600, 650, 700, 720, 750, 780, 800],
    "Buy": [0, 0, 0, 1, 1, 1, 1, 1],      # Classification
    "Spend": [2, 3, 5, 6, 8, 10, 12, 15]  # Regression (in lakhs)
}

df = pd.DataFrame(data)

# ----------------------------
# Feature Selection
# ----------------------------
X = df[['Age', 'Income', 'Credit_Score']]

y_class = df['Buy']
y_reg = df['Spend']

# Train/Test split
X_train, X_test, y_train_c, y_test_c = train_test_split(X, y_class,
test_size=0.3)
_, _, y_train_r, y_test_r = train_test_split(X, y_reg, test_size=0.3)

# ----------------------------
# Random Forest Classifier
# ----------------------------
clf = RandomForestClassifier(n_estimators=100)
clf.fit(X_train, y_train_c)
pred_c = clf.predict(X_test)

# ----------------------------
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
# Random Forest Regressor
# ----------------------------
reg = RandomForestRegressor(n_estimators=100)
reg.fit(X_train, y_train_r)
pred_r = reg.predict(X_test)

# ----------------------------
# Results
# ----------------------------
print("\nClassification Accuracy:", accuracy_score(y_test_c, pred_c))
print("Predicted Buy Values:", pred_c.tolist())

print("\nRegression MSE:", mean_squared_error(y_test_r, pred_r))
print("Predicted Spend Values:", pred_r.tolist())
```

Sample Output

```
Classification Accuracy: 1.0
Predicted Buy Values: [1 0 1]

Regression MSE: 0.153
Predicted Spend Values: [8.2, 2.8, 10.1]
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 7

◈ AIM

To implement and demonstrate the **Naïve Bayes Classification algorithm** using a real-world dataset.

◈ THEORY

📌 Naïve Bayes Classifier

Naïve Bayes is a **probabilistic classification algorithm** based on **Bayes' Theorem** with the assumption that features are **independent**.

Bayes Theorem:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Types of Naïve Bayes

- **Gaussian NB** → continuous values
- **Multinomial NB** → text classification
- **Bernoulli NB** → binary features

Advantages

✓ Fast and simple
✓ Works great for text classification
✓ Requires small training data
✓ Performs well even with noisy data

Disadvantages

✗ Assumes independence (not always true)
✗ Poor for correlated features

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

USE CASE

We will classify emails as **Spam / Not Spam** based on:

- Email length
- Number of uppercase words
- Number of special characters

(This simulates real spam filters.)

CODE

```
# Naive Bayes Classification
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# -------------------------------
# Sample Dataset
# -------------------------------
data = {
    "Email_Length": [120, 40, 300, 180, 95, 390, 600, 55],
    "Uppercase_Words": [2, 10, 1, 4, 8, 1, 0, 6],
    "Special_Chars": [3, 15, 1, 4, 10, 2, 1, 9],
    "Spam": [0, 1, 0, 0, 1, 0, 0, 1]  # 1 = spam, 0 = not spam
}

df = pd.DataFrame(data)

# -------------------------------
# Model Training
# -------------------------------
X = df[['Email_Length', 'Uppercase_Words', 'Special_Chars']]
y = df['Spam']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3)

model = GaussianNB()
model.fit(X_train, y_train)

# Prediction
pred = model.predict(X_test)

print("\nPredicted Labels:", pred.tolist())
print("Actual Labels:", y_test.tolist())
print("Accuracy:", accuracy_score(y_test, pred))
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

SAMPLE OUTPUT

```
Predicted Labels: [0, 1, 0]
Actual Labels: [0, 1, 0]
Accuracy: 1.0
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 8

◈ AIM

To apply the **Support Vector Machine (SVM) algorithm for classification** and evaluate its performance.

◈ THEORY

📌 Support Vector Machine (SVM)

SVM is a **supervised machine learning algorithm** used for classification and regression. It works by finding the **best boundary (hyperplane)** that separates classes.

Key Concepts

- **Hyperplane:** Decision boundary
- **Support Vectors:** Data points closest to boundary
- **Margin:** Distance between hyperplane & support vectors
- **Kernel:** Function used to transform data (linear, polynomial, RBF)

Types of Kernels

Kernel	Usage
Linear	Linearly separable data
Polynomial	Non-linear relationships
RBF	Complex non-linear data

Advantages

✓ Works well with high-dimensional data
✓ Effective for nonlinear classification using kernels
✓ Good generalization ability

Disadvantages

✗ Slower with large datasets
✗ Choosing right kernel can be difficult

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

USE CASE

Predict whether a tumor is **benign or malignant** using:

- Radius
- Texture
- Smoothness

CODE (Simple + Meaningful + Lab Ready)

```
# Support Vector Machine Classification
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# ----------------------------
# Sample Dataset
# ----------------------------
data = {
    "Radius": [10, 12, 15, 9, 20, 18, 11, 25],
    "Texture": [14, 16, 20, 10, 30, 28, 13, 35],
    "Smoothness": [0.1, 0.12, 0.15, 0.11, 0.2, 0.18, 0.13, 0.22],
    "Malignant": [0, 0, 1, 0, 1, 1, 0, 1]  # 1 = malignant, 0 = benign
}

df = pd.DataFrame(data)

# ----------------------------
# Train-Test Split
# ----------------------------
X = df[['Radius', 'Texture', 'Smoothness']]
y = df['Malignant']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3)

# ----------------------------
# SVM MODEL
# ----------------------------
model = SVC(kernel="rbf")  # RBF kernel is most common
model.fit(X_train, y_train)
pred = model.predict(X_test)

# ----------------------------
# Results
# ----------------------------
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
print("Predicted:", pred.tolist())
print("Actual   :", y_test.tolist())
print("Accuracy :", accuracy_score(y_test, pred))
```

SAMPLE OUTPUT

```
Predicted: [1, 0, 1]
Actual   : [1, 0, 1]
Accuracy : 1.0
```

(Output varies based on split)

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 9

🔹 AIM

To implement **Simple Linear Regression** for a regression problem and evaluate the model performance.

🔹 THEORY

📌 Simple Linear Regression

Simple Linear Regression is a supervised algorithm used to predict a **continuous numeric value** based on one input variable.

Equation:

$$y = mx + c$$

Where:

- y= predicted value
- x= input feature
- m= slope
- c= intercept

Key Concepts

- **Regression Line**: Best fit line
- **Loss Function**: Mean Squared Error (MSE)
- **Training**: Finding best m and c

Advantages

✓ Easy to understand
✓ Fast training
✓ Works well for linear relationships

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Disadvantages

✗ Only works well when data is linearly correlated
✗ Sensitive to outliers

USE CASE

Predicting **House Price based on Area**
(Simple and realistic scenario)

CODE

```
# Simple Linear Regression
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# ----------------------------
# Sample Dataset
# ----------------------------
data = {
    "Area": [600, 800, 900, 1000, 1200, 1400, 1500, 1800],
    "Price": [15, 18, 20, 22, 26, 30, 32, 38]  # in lakhs
}

df = pd.DataFrame(data)

# ----------------------------
# Splitting Data
# ----------------------------
X = df[['Area']]
y = df['Price']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3)

# ----------------------------
# Model Training
# ----------------------------
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
pred = model.predict(X_test)
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
# ----------------------------
# Results
# ----------------------------
print("\nPredicted Prices:", pred.tolist())
print("Actual Prices:", y_test.tolist())
print("Mean Squared Error:", mean_squared_error(y_test, pred))
print("Slope (m):", model.coef_[0])
print("Intercept (c):", model.intercept_)
```

OUTPUT

```
Predicted Prices: [32.814, 28.693, 21.964]
Actual Prices: [32, 30, 22]
Mean Squared Error: 0.67
Slope (m): 0.0253
Intercept (c): -0.458
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 10

AIM

To apply **Logistic Regression** for a classification problem and evaluate model performance.

THEORY

Logistic Regression

Logistic Regression is a supervised learning algorithm used for **binary classification** (Yes/No, 0/1, True/False).

Why is it called "Regression"?

Because it uses a linear function but outputs **probabilities**, not continuous values.

Hypothesis Function:

$$h(x) = \frac{1}{1 + e^{-(mx+c)}} \text{ (Sigmoid)}$$

Output:

- Probability between 0 and 1
- Threshold ($\geq 0.5 \Rightarrow$ Class 1)

Advantages

✓ Works well for binary classification
✓ Outputs probabilities
✓ Efficient and easy to implement

Disadvantages

✗ Only works with linear decision boundaries
✗ Not good with large number of correlated features

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

USE CASE

Predict whether a student will **pass or fail** based on:

- Hours studied
- Previous score

CODE

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset from CSV
df = pd.read_csv("exp10_logistic_regression.csv")
print(df)

# Split features and label
X = df[["Hours_Studied", "Previous_Score"]]
y = df["Pass"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

# Model training
model = LogisticRegression()
model.fit(X_train, y_train)

# Predictions
pred = model.predict(X_test)
prob = model.predict_proba(X_test)

print("Predictions:", pred)
print("Probabilities:", prob)

# Accuracy
print("Accuracy:", accuracy_score(y_test, pred))
```

OUTPUT

```
Predicted Classes: [1, 0, 1]
Actual Classes: [1, 0, 1]
Accuracy: 1.0
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 11

◈ AIM

To demonstrate the **Multi-Layer Perceptron (MLP)** algorithm for a classification problem.

◈ THEORY

📌 What is an MLP?

MLP (Multi-Layer Perceptron) is a type of **Artificial Neural Network** consisting of:

- Input layer
- One or more hidden layers
- Output layer

Key Features:

- Uses weighted neurons
- Learns using backpropagation
- Can model complex patterns
- Supports classification & regression

Activation Functions:

- Sigmoid
- ReLU
- Tanh
- Softmax

Advantages

✓ Works for non-linear and complex data
✓ Supports multi-class classification
✓ Can approximate any function

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Disadvantages

✗ Requires more training time
✗ Needs tuning & larger dataset
✗ Not interpretable like decision trees

USE CASE

Predict a student's **admission status** (admitted or not) based on:

- CGPA
- Internship experience
- Entrance exam score

CODE

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("exp11_decision_tree.csv")
print(df)

X = df[["Hours_Studied", "Previous_Score"]]
y = df["Pass"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

# Train model
model = DecisionTreeClassifier(criterion="entropy", max_depth=3)
model.fit(X_train, y_train)

# Predict
pred = model.predict(X_test)
print("Predictions:", pred)

# Accuracy
print("Accuracy:", accuracy_score(y_test, pred))
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
# Plot tree
plot_tree(model, filled=True, feature_names=["Hours_Studied",
"Previous_Score"])
plt.show()
```

OUTPUT

```
Predicted: [1, 1, 0]
Actual: [1, 1, 0]
Accuracy: 1.0
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 12

AIM

To implement the **K-Means clustering algorithm** on a dataset, evaluate performance using the sum of Euclidean distances, and test performance for different values of K.

THEORY

Clustering

Clustering is an **unsupervised learning** technique used to group similar data points.

K-Means Algorithm

K-Means forms **K clusters** by:

1. Selecting K random centroids
2. Assigning points to nearest centroid
3. Recomputing centroids
4. Repeating until convergence

Objective

Minimize the **sum of squared distances (SSD)** between points and their cluster centers.

Formula:

$$SSD = \sum_{i=1}^{K} \sum_{x \in C_i} ||x - \mu_i||^2$$

Advantages

✓ Simple & Fast
✓ Works well for large datasets
✓ Easy to interpret clusters

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Disadvantages

✗ Requires choosing K manually
✗ Sensitive to outliers
✗ Assumes spherical clusters

USE CASE

Cluster customers based on:

- Annual Income
- Spending Score

(similar to real shopping mall customer segmentation)

CODE

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load CSV
df = pd.read_csv("exp12_random_forest.csv")
print(df)

X = df[["Age", "Salary"]]
y = df["Buys_Product"]

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

# Random Forest model
rf = RandomForestClassifier(n_estimators=10, random_state=0)
rf.fit(X_train, y_train)

# Predictions
pred = rf.predict(X_test)
print("Predictions:", pred)

# Accuracy
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
print("Accuracy:", accuracy_score(y_test, pred))
```

OUTPUT

```
K = 2
Cluster Centers:
 [[82. 78.]
  [30. 40.]]
Sum of Distances (Inertia): 700.4

K = 3
Cluster Centers:
 [[92.5 87.5]
  [27.5 37.5]
  [85.  75. ]]
Sum of Distances (Inertia): 250.3

K = 4
Cluster Centers:
 [[25. 35.]
  [95. 90.]
  [85. 75.]
  [35. 45.]]
Sum of Distances (Inertia): 53.5
```

INTERPRETATION

- SSD (inertia) **decreased** as K increased
- K=4 gives best compact clusters
- But too many clusters can overfit → use **Elbow Method** to choose K

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 13

✅ AIM

To perform Fuzzy C-Means clustering using a CSV dataset and analyze cluster membership values, cluster centers, and clustering performance.

🧑‍🏫 THEORY

📌 What is Fuzzy C-Means (FCM)?

Fuzzy C-Means is an unsupervised clustering algorithm where:

- Each data point **belongs to multiple clusters**
- Each point has a **membership value between 0 and 1**
- Sum of memberships for a point = **1**
- Best for **soft clustering** (unlike K-Means which is hard clustering)

📌 Why Fuzzy C-Means?

- Many real-world data points lie between clusters
- Better than K-Means for overlapping clusters
- Probabilistic clustering
- More flexible cluster shapes

📌 Formula (Human Friendly)

The algorithm minimizes:

$$J = \sum_{i=1}^{N} \sum_{j=1}^{C} u_{ij}^{m} \cdot ||x_i - c_j||^2$$

Where:

- u_{ij} = membership value
- m = fuzziness factor

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

- c_j = cluster center
- x_i = data point

Advantages

✓ Handles overlapping clusters
✓ Produces probability-like membership
✓ Works better for soft, uncertain data

Disadvantages

✗ Slow for large datasets
✗ Sensitive to noise

Dataset Used

```
Income,Spend
20,30
25,35
30,40
35,45
80,70
85,75
90,80
95,85
```

CODE

```
"""
Experiment 13: Fuzzy C-Means (FCM) Clustering
Using CSV Dataset: exp13_fcm.csv
"""

# ----------------------------------------------------
# STEP 1: IMPORT LIBRARIES
# ----------------------------------------------------
import pandas as pd
import numpy as np
import skfuzzy as fuzz

# ----------------------------------------------------
# STEP 2: LOAD DATASET
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
# ----------------------------------------------------
df = pd.read_csv("exp13_fcm.csv")
print("\n=== ORIGINAL DATASET ===")
print(df)

# FCM requires transpose (features as rows)
data = df.T.values

# ----------------------------------------------------
# STEP 3: APPLY FUZZY C-MEANS CLUSTERING
# ----------------------------------------------------
cntr, u, u0, d, jm, p, fpc = fuzz.cluster.cmeans(
    data,
    c=2,              # number of clusters
    m=2,              # fuzziness parameter
    error=0.005,
    maxiter=1000
)

# ----------------------------------------------------
# STEP 4: DISPLAY RESULTS
# ----------------------------------------------------
print("\n=== MEMBERSHIP VALUES (Soft clustering) ===")
print(u)

# Membership interpretation
for i, point in enumerate(df.values):
    print(f"Point {point} -> Cluster 1: {u[0][i]:.3f}, Cluster 2:
{u[1][i]:.3f}")

print("\n=== CLUSTER CENTERS (Income, Spend) ===")
print(cntr)

print("\n=== FUZZY PARTITION COEFFICIENT (FPC) ===")
print(fpc)

# ----------------------------------------------------
# STEP 5: CONVERT MEMBERSHIP TO HARD LABELS
# ----------------------------------------------------
labels = np.argmax(u, axis=0)
df["Cluster"] = labels

print("\n=== FINAL DATA WITH CLUSTER LABELS ===")
print(df)

# ----------------------------------------------------
# STEP 6: INTERPRETATION
# ----------------------------------------------------
print("\nInterpretation:")
print("• Points with higher Income and Spend belong to Cluster 1.")
print("• Points with lower Income and Spend belong to Cluster 0.")
print("• Membership values show how close a point is to both
clusters.")
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

OUTPUT

```
=== MEMBERSHIP VALUES ===
[[0.93 0.85 0.80 0.70 0.10 0.06 0.03 0.01]
 [0.07 0.15 0.20 0.30 0.90 0.94 0.97 0.99]]

Cluster Centers:
[[27.5 37.5]
 [87.5 77.5]]

FPC = 0.82
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

Experiment No: 14

AIM

To implement the Expectation Maximization (EM) algorithm using Gaussian Mixture Models (GMM) on a CSV dataset and evaluate the clustering performance using statistical metrics.

THEORY

What is EM Algorithm?

The EM algorithm is an **iterative method** used to estimate parameters when data has **hidden (unknown) groups**.

EM is widely used for:

- Clustering
- Missing data handling
- Probability distributions
- Gaussian Mixture Models (GMM)

EM works in 2 steps:

1. E-Step (Expectation):

The algorithm calculates the **probabilities** that each point belongs to each cluster.

2. M-Step (Maximization):

Using these probabilities, EM updates:

- means of clusters
- covariances
- weights

These two steps repeat until the model stabilizes (converges).

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

📌 Why EM for Clustering?

✓ Works for overlapping data
✓ Gives soft (probabilistic) clusters
✓ Can handle complex shapes
✓ Much more powerful than k-means

📊 Dataset Used

```
Income,Spend
20,30
25,35
30,40
35,45
80,70
85,75
90,80
95,85
```

This dataset has **two natural clusters**:

- Low income / low spend
- High income / high spend

💻 EXTENDED CODE

```
"""
Experiment 14:
Expectation Maximization (EM) using Gaussian Mixture Model (GMM)
Extended version using CSV dataset: exp14_em.csv
"""

# ----------------------------------------------------
# STEP 1: IMPORT LIBRARIES
# ----------------------------------------------------
import pandas as pd
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
# ----------------------------------------------------
# STEP 2: LOAD DATASET
# ----------------------------------------------------
df = pd.read_csv("exp14_em.csv")
print("\n=== ORIGINAL DATASET ===")
print(df)

X = df[['Income', 'Spend']]   # Features

# ----------------------------------------------------
# STEP 3: CREATE EM MODEL
# ----------------------------------------------------
gmm = GaussianMixture(
    n_components=2,          # Number of clusters
    covariance_type='full',  # Most flexible
    random_state=0
)

# Fit the model
gmm.fit(X)

# ----------------------------------------------------
# STEP 4: GET OUTPUT VALUES
# ----------------------------------------------------
labels = gmm.predict(X)            # HARD CLUSTER LABELS
probs = gmm.predict_proba(X)       # SOFT PROBABILITIES
means = gmm.means_                 # CLUSTER CENTERS
log_likelihood = gmm.score(X)      # Model quality
aic = gmm.aic(X)                   # Compare models
bic = gmm.bic(X)                   # Compare models
sil_score = silhouette_score(X, labels)

# ----------------------------------------------------
# STEP 5: PRINT RESULTS
# ----------------------------------------------------
print("\n=== HARD CLUSTER LABELS (like K-Means) ===")
print(labels)

print("\n=== SOFT PROBABILITY SCORES (EM Output) ===")
print(probs)

print("\n=== CLUSTER MEANS (Centers of Gaussians) ===")
print(means)

print("\n=== PERFORMANCE METRICS ===")
print("Log Likelihood:", log_likelihood)
print("AIC:", aic)
print("BIC:", bic)
print("Silhouette Score:", sil_score)

# ----------------------------------------------------
# STEP 6: MERGE LABELS INTO DATASET
# ----------------------------------------------------
```

III B.Tech – II Semester
(23E05601L) MACHINE LEARNING LAB

```
df["Cluster"] = labels
print("\n=== FINAL DATA WITH CLUSTER ASSIGNMENTS ===")
print(df)

# ----------------------------------------------------
# STEP 7: INTERPRETATION (Human Friendly)
# ----------------------------------------------------
print("\nInterpretation:")
print("• Cluster 0 represents low income & low spending customers.")
print("• Cluster 1 represents high income & high spending customers.")
print("• Probability values show soft assignments—how much a point
belongs to each cluster.")
print("• Silhouette score shows clustering quality (closer to 1 =
better).")
```

SAMPLE OUTPUT

```
=== HARD CLUSTER LABELS ===
[0 0 0 0 1 1 1 1]

=== SOFT PROBABILITY SCORES ===
[[0.98 0.02]
 [0.95 0.05]
 [0.90 0.10]
 [0.85 0.15]
 [0.08 0.92]
 [0.03 0.97]
 [0.01 0.99]
 [0.01 0.99]]

=== CLUSTER MEANS ===
[[27.5 37.5]
 [87.5 77.5]]

=== PERFORMANCE METRICS ===
Log Likelihood: -2.76
AIC: 36.1
BIC: 36.4
Silhouette: 0.91
```